Design a Banking Loan
Design a Ticketing Ticket Service System
Design a Uber-like Search System
Design a Ticketing Ticket Service System
Design a Banking Loan
Design a Shopping Cart System
Design a Uber-like Search System
Design a Ride Sharing App
Design a Shopping Cart System
Design a Shopping Cart System
Design an File System
Design an Online Banking and Transaction System

09
Design a Grocery Store System

In this chapter, we will explore the design of a grocery store system. This system is tailored for grocery store workers to strategically operations like managing the line, calculating pricing, and applying discounts.

Grocery Store System

Requirements Gathering

Here is an example of a typical prompt and an ideal response:

"Imagine you're at a grocery store, filling your cart with fresh produce, snacks, and household essentials. At the checkout, the cashier rings up your purchase, calculates the total, and applies any discounts or coupons you have. Now, picture a system that streamlines this entire process, from managing the line to calculating prices and applying discounts. Your task is to design a grocery store system that does all of this."

Requirements clarification

Here is an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: What are the primary operations the grocery store system needs to support?

Interviewer: The system should support cashiers, including displayed handlers and customers, in managing the item catalog, tracking inventory, and processing transactions with accurate pricing and applicable discounts.

Candidate: I'd like to confirm my understanding of the checkout process. The cashier scans or enters a code for each item, and the system displays the price. The customer then pays, and the system updates inventory. Is there anything else that happens during the checkout process? For example, do we need to handle returns or exchanges, or apply loyalty points or coupons? Instead, it sounds like generalized Grocery Store system, not just a checkout system.

Interviewer: That's an accurate understanding of the system.

Candidate: Regarding the item catalog, what kinds of pricing rules do we need to support? Do we need to support bulk pricing, discounts for specific customers, or time-sensitive promotions?

Interviewer: The item catalog should support bulk pricing, discounts for specific customers, and flash promotions, etc.?

Candidate: For discounts, can I assume it works like Uber? The system should track discount campaigns, which can apply to specific items or categories. It should dynamically apply the best rates. The system should automatically apply the highest discount?

Interviewer: That sounds good.

Requirements

This question has multiple components, so grouping similar ones makes it easier to manage and track. The requirements can be broken down into three groups:

Catalog management

- Admins can add, update, and remove items from the catalog.
- The catalog stores item details, including name, category, price, and barcode.

Inventory management

- Prevent transfers can update inventory when shipments arrive.
- The system automatically updates inventory when items are sold.

Checkout process

- Cashiers can scan barcodes or manually enter item codes to build an order.
- Customers can see a breakdown of the entire order, including items, discounts, and the subtotal.
- The system automatically applies relevant discounts automatically.
- The system calculates the final price, including taxes, handle exceptions, and calculate change.
- A receipt is issued upon completion.

Discount management

- Admins can create discount campaigns for specific items or categories.
- Flexible discounts apply to both fixed amounts (dollars off) and percentages.

Here is the list of requirements that we have:

- The system should provide clear, user-friendly error messages (e.g., for invalid barcodes or insufficient inventory) for the cashier.
- The system's performance (scanning, inventory, checkout, discounts) must be reliable to allow cashiers to process customer transactions efficiently.
- The system should support concurrent transactions without performance issues.

Identify Core Objects

Before diving into the design, it's important to enumerate the core objects.

- **Item**: Represents a product in the catalog, including its name, corresponding Meals such as name, barcode, category, and price.
- **Catalog**: Acts as the central repository for all items, managing the collection of products and supporting operations like adding, removing, and searching for items.
- **Inventory**: Tracks the stock levels for each item. It updates the count of available items when sales occur or new shipments are received, ensuring accurate stock management.
- **Discount**: Defines the rules for applying discounts, including the conditions (such as specific items or categories) and the actual discount amount (fixed amount or percentage).
- **DiscountCriteria**: Specifies the conditions under which a discount applies, such as the items in the order, quantities, and specific customers or categories. This criteria is used to determine if a discount is applicable.
- **DiscountCalculationStrategy**: Implements different discount calculation strategies, allowing for flexibility in calculating discounts based on various criteria.

Design Class Diagram

Now that we know the core objects and their roles, the next step is to create classes and methods that turn this requirements into an easy-to-maintain system. Let's take a closer look.

Item

The first component in our class diagram is the Item class, which represents individual products in the store. It encapsulates attributes like name, barcode, category, and price.

Below is the representation of the class:

Catalog

The Catalog class is responsible for maintaining a structured list of all available products, each uniquely identified by a barcode. It provides methods to add, update, remove, and retrieve items.

The UML diagram below illustrates this structure:

Inventory

Building on the Catalog class, the Inventory class is a critical component of the grocery store system, responsible for managing stock levels of items. It maintains a mapping between each item's barcode and its corresponding stock quantity.

Here is the representation of the class:

Discount

Moving to the Discount class, this component manages various types of discounts, including fixed amount discounts or percentage-based discounts. It includes attributes for specifying applicable criteria, such as specific items or categories, and the actual discount amount.

The UML diagram below illustrates this structure:

DiscountCriteria

The DiscountCriteria class encapsulates the logic to determine whether a discount applies to an item. It provides a flexible, extensible framework for defining applicability checks, such as item-based and category-based criteria, allowing the system to support diverse discount rules and promotions.

The UML diagram below illustrates this structure:

DiscountCriteria Implementations

DiscountCalculationStrategy

The DiscountCalculationStrategy interface encapsulates the logic for calculating discounts. It uses the Strategy Pattern to provide an easy-to-maintain system that can handle various discount calculations.

Below is the representation of the interface:

DiscountCalculationStrategy Interface and Concrete Classes

AmountBasedDiscount: This strategy applies a fixed discount amount to the original price. For example, if the original price is $100 and the fixed discount amount is $10, the final price after the discount will be $90. This approach is straightforward and ideal for scenarios offering consistent reductions, such as seasonal or clearance discounts.

PercentageBasedDiscount: This strategy applies a percentage-based discount to the original price. For instance, if the original price is $100 and the discount percentage is 10%, the price after the discount will be $90. This approach is more flexible and suitable for campaigns offering proportional reductions, such as seasonal or category-based discounts.

Design choice: Using separate classes for each discount calculation strategy ensures adherence to the Open/Closed Principle. This design allows for easy extensibility, enabling the addition of new discount calculations without modifying the existing implementations, adhering to the Open/Closed Principle.

DiscountCampaign

Finally, we have the DiscountCampaign class, which models active discount campaigns and is a key component for applying discounts to items. This class encapsulates the logic to encapsulate different discount campaigns, including specifications for the amount discounts, ensuring flexibility in how discounts are computed. The class uses a DiscountCriteria interface to specify which items the campaign applies to, and it uses a DiscountCalculationStrategy instance to perform the actual discount calculation. This design promotes a flexible and maintainable approach to managing various discount campaigns.

The UML diagram below illustrates this structure:

Design choice: The DiscountCriteria interface separates item-level details from the higher-level order. This ensures that each criteria's specificity and price logic are encapsulated, enhancing the design modular and easier to maintain.

Order

The Order class represents an active transaction during the checkout process. It tracks the list of items in order, along with pricing and applied discounts. The class provides methods to calculate the subtotal before discounts and the total amount after discounts.

Below is the representation of this class:

Design choice: The Order class focuses on managing transactional data for the checkout process. It delegates the handling of individual item specifics to the Checkout class, ensuring a clean separation of responsibilities.

Receipt

The Receipt class is the final record of a completed transaction, consolidating all relevant details into a formatted output for the customer. It includes essential information such as the order summary, payment details, and any changes to be given to the customer.

Here is the representation of this class:

Checkout

Moving on to the Checkout class, which encapsulates the logic for handling the checkout process within the grocery store system. It oversees an active Order object to track the transaction details. Additionally, it applies active discount campaigns to calculate the final prices for the customer's purchase.

Below is the representation of this class:

Design choice: The Checkout class focuses solely on managing transactional data for the checkout process. It delegates the handling of individual item specifics to the Checkout class, ensuring a clean separation of responsibilities.

Complete Class Diagram

Below is the complete class diagram of our grocery store system. The detailed methods and attributes are skipped to make the diagram more readable.

Class Diagram of Grocery Store System

Code - Grocery Store System

In this section, we'll implement the core functionality of the grocery store system, focusing on key areas such as managing products and inventory, and streamlining the checkout process, including discount handling.

Item

We implement the Item class, which encapsulates product attributes, including the name, barcode, category, and price. It is designed as a simple data container with no business logic, making it easy to use and maintain.

```
public class Item {
    private String name;
    private String barcode;
    private Float price;
    private Float tax;
    private String category;

    public Item(String name, String barcode, Float price) {
        this.name = name;
        this.barcode = barcode;
        this.price = price;
        this.tax = 0.0f;
        this.category = null;
    }

    // getter and setter methods are omitted for brevity.
}
```

Catalog

We define the Catalog class, which acts as the centralized repository for managing the store's item inventory. Below is the implementation of the class:

```
public class Catalog {
    // Map to store items, using barcode as the key.
    private Map<String, Item> items = new HashMap<>();

    public void updateItem(Item item) {
        items.put(item.getBarcode(), item);
    }

    public void removeItem(String barcode) {
        items.remove(barcode);
    }

    public Item getItem(String barcode) {
        return items.get(barcode);
    }
}
```

Implementation notes: This class uses a HashMap to store items, using barcodes as the keys. The hash table provides efficient insertion, updates, and deletions.

Inventory

The Inventory class manages the stock levels of items, ensuring accurate tracking of inventory levels to facilitate operations. It provides methods to handle stock operations, such as adding new stock, decreasing stock for sales, and querying the current stock levels for a specific item.

Here is the implementation of the class:

```
public class Inventory {
    // Map to store stock levels, using barcode as the key.
    private Map<String, Integer> stock = new HashMap<>();

    public void addStock(String barcode, int count) {
        stock.put(barcode, stock.getOrDefault(barcode, 0) + count);
    }

    public void removeStock(String barcode, int count) {
        stock.put(barcode, stock.getOrDefault(barcode, 0) - count);
    }

    public int getStock(String barcode) {
        return stock.getOrDefault(barcode, 0);
    }
}
```

- **addStock(String barcode, int count)**: Increases the stock for the specified barcode.
- **removeStock(String barcode, int count)**: Decreases the stock for the specified barcode.
- **getStock(String barcode)**: Returns the current stock level for the specified barcode.

Implementation notes: The Inventory class uses a HashMap to store stock levels, with barcodes serving as the keys. This ensures efficient updates and queries. This supports efficient inventory management during sales and shipments, which is crucial for maintaining accurate stock levels in real-time.

DiscountCampaign

The DiscountCampaign class models promotional rules for specific items or categories. It separates the logic for determining discount applicability (using DiscountCriteria) from the calculation of discount values (using DiscountCalculationStrategy). This design enhances flexibility and extensibility for various discount configurations.

Below is the implementation of this class:

```
public class DiscountCampaign {
    private String campaignId;
    private String description;
    private DiscountCriteria criteria;
    private DiscountCalculationStrategy calculationStrategy;

    // Constructor to initialize the discount campaign.
    public DiscountCampaign(String campaignId, String description,
                            DiscountCriteria criteria,
                            DiscountCalculationStrategy calculationStrategy) {
        this.campaignId = campaignId;
        this.description = description;
        this.criteria = criteria;
        this.calculationStrategy = calculationStrategy;
    }

    // Check if the discount applies to the given item.
    public boolean isApplicable(Item item) {
        return criteria.isApplicable(item);
    }

    // Calculate the discounted price for the given order.
    public BigDecimal calculateDiscount(BigDecimal originalPrice) {
        return calculationStrategy.calculateDiscount(originalPrice);
    }

    // getter and setter methods are omitted for brevity.
}
```

isApplicable(Item item): Checks if the discount applies to a given item based on the defined criteria. Returns true if applicable, otherwise, false.

calculateDiscount(BigDecimal price): Computes the discount amount using the specified calculationStrategy. Supports both fixed amount and percentage-based discounts.

Implementation notes: The DiscountCampaign class uses composition to hold a single DiscountCriteria and DiscountCalculationStrategy. This design allows for easy extension by implementing new criteria or calculation strategies without modifying the core campaign logic.

Note for brevity, the code for DiscountCriteria and DiscountCalculationStrategy is omitted. These interfaces define applicability checks (e.g., category or item-based criteria) and discount calculations (e.g., percentage-based or fixed amount).

Order

The Order class tracks individual product orders in-store, along with its quantity, it calculates the total cost for the item by multiplying its unit price by the quantity, enabling seamless integration with the order's subtotal and total calculations.

Here is the implementation of this class:

```
public class Order {
    // Unique identifier for the order.
    private String orderID;
    // List of items in the order.
    private List<Item> items = new ArrayList<>();
    // Map of item to its quantity.
    private Map<Item, Integer> quantities = new HashMap<>();
    // Applied discounts.
    private List<DiscountCampaign> appliedDiscounts = new ArrayList<>();
    // Base price before any discounts.
    private BigDecimal subTotal = BigDecimal.ZERO;

    // Add single item to the order.
    public void addItem(Item item) {
        items.add(item);
        quantities.put(item, quantities.getOrDefault(item, 0) + 1);
        updateSubTotal();
    }

    // Add multiple items to the order.
    public void addItem(List<Item> items, int quantity) {
        for (Item item : items) {
            addItem(item, quantity);
        }
    }

    // Calculate the subtotal of all items ordered.
    public BigDecimal getSubTotal() {
        return items.stream()
                .map(item -> {
                    BigDecimal price = item.getPrice();
                    int quantity = quantities.get(item);
                    return price.multiply(BigDecimal.valueOf(quantity));
                })
                .reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    // Apply a specific discount campaign to the order.
    public void applyDiscount(DiscountCampaign discount, DiscountCriteria criteria) {
        if (criteria.isApplicable(this.getItems())) {
            appliedDiscounts.add(discount);
            updateSubTotal();
        }
    }

    // Apply a discount with custom calculation strategy.
    public void applyDiscount(DiscountCampaign discount,
                              DiscountCriteria criteria,
                              DiscountCalculationStrategy calculationStrategy) {
        if (criteria.isApplicable(this.getItems())) {
            appliedDiscounts.add(discount);
            updateSubTotal(calculationStrategy);
        }
    }

    // Apply a discount.
    public void applyDiscount(DiscountCampaign discount) {
        if (discount.isApplicable(this.getItems())) {
            appliedDiscounts.add(discount);
            updateSubTotal();
        }
    }

    // Update the subtotal after applying discounts.
    private void updateSubTotal() {
        subTotal = getSubTotal();
        for (DiscountCampaign discount : appliedDiscounts) {
            subTotal = subTotal.subtract(discount.calculateDiscount(subTotal));
        }
    }

    // Update the subtotal using a specific calculation strategy.
    private void updateSubTotal(DiscountCalculationStrategy calculationStrategy) {
        subTotal = getSubTotal();
        for (DiscountCampaign discount : appliedDiscounts) {
            subTotal = subTotal.subtract(calculationStrategy.calculateDiscount(subTotal));
        }
    }

    // getter and setter methods are omitted for brevity.
}
```

applyDiscount(DiscountCampaign discount, DiscountCriteria criteria): These methods apply the item's total price after applying a given discount campaign, instead of simply decreasing the base price until it hits zero. Crucially, it factors in the discount to determine the applicable amount.

applyDiscount(DiscountCampaign discount, DiscountCriteria criteria, DiscountCalculationStrategy calculationStrategy): This method provides maximum flexibility by allowing a specific calculation strategy to be used for the discount.

updateSubTotal(): These methods recalculate the subtotal after applying discounts. The method computes the item's total price after applying a given discount campaign, instead of simply decreasing the base price until it hits zero. Crucially, it factors in the discount to determine the applicable amount.

Checkout

The Checkout class manages customer transactions by maintaining a list of purchased items and their associated discounts. It provides methods to add items to the order, apply discounts, and calculate the final amount. The class ensures accurate transaction tracking and leverages the order's calculation capabilities.

```
public class Checkout {
    // Current order being processed.
    private Order currentOrder;
    // List of active discount campaigns.
    private List<DiscountCampaign> activeDiscounts = new ArrayList<>();

    // Start a new order.
    public void startNewOrder() {
        currentOrder = new Order();
        activeDiscounts.clear();
    }

    // Add item to the current order.
    public void addItemToOrder(Item item, int quantity) {
        if (currentOrder == null) {
            startNewOrder();
        }
        currentOrder.addItem(item, quantity);
        applyDiscountsToCurrentOrder();
    }

    // Calculate the final price after all discounts.
    public BigDecimal getFinalTotal() {
        if (currentOrder == null) {
            return BigDecimal.ZERO;
        }
        return currentOrder.getSubTotal();
    }

    // Apply multiple discounts.
    public void applyDiscounts(List<DiscountCampaign> activeDiscounts) {
        this.activeDiscounts.addAll(activeDiscounts);
        applyDiscountsToCurrentOrder();
    }

    // Get instance.
    public Checkout getInstance() {
        return this;
    }

    // Add active discount.
    public void addActiveDiscount(DiscountCampaign discount) {
        activeDiscounts.add(discount);
        applyDiscountsToCurrentOrder();
    }

    // Add multiple active discounts.
    public void addActiveDiscounts(List<DiscountCampaign> discounts) {
        activeDiscounts.addAll(discounts);
        applyDiscountsToCurrentOrder();
    }

    // Remove active discount.
    public void removeActiveDiscount(DiscountCampaign discount) {
        activeDiscounts.remove(discount);
        applyDiscountsToCurrentOrder();
    }

    // Clear all active discounts.
    public void clearActiveDiscounts() {
        activeDiscounts.clear();
        applyDiscountsToCurrentOrder();
    }

    // Apply discounts to current order.
    private void applyDiscountsToCurrentOrder() {
        if (currentOrder == null) {
            return;
        }
        for (DiscountCampaign discount : activeDiscounts) {
            currentOrder.applyDiscount(discount);
        }
    }

    // Apply single discount to current order.
    private void applyDiscountToCurrentOrder(DiscountCampaign discount) {
        if (currentOrder == null) {
            return;
        }
        currentOrder.applyDiscount(discount);
    }
}
```

startNewOrder(): Initializes a new order and clears any previous discounts.

addItemToOrder(Item item, int quantity): Adds items to the current order and applies relevant discounts.

getFinalTotal(): Returns the final price after all discounts have been applied.

applyDiscounts(List<DiscountCampaign> activeDiscounts): Applies a list of discount campaigns to the current order.

addActiveDiscount(DiscountCampaign discount): Adds a single discount campaign to the active discounts.

removeActiveDiscount(DiscountCampaign discount): Removes a specific discount campaign from the active discounts.

clearActiveDiscounts(): Clears all active discount campaigns.

applyDiscountsToCurrentOrder(): Applies all active discounts to the current order.

applyDiscountToCurrentOrder(DiscountCampaign discount): Applies a single discount to the current order.

Implementation notes: The Checkout class uses a singleton pattern to ensure consistency across the system. It automatically applies active discounts when items are added to the order, ensuring real-time price calculations. The class maintains a clean separation between order management and discount application.

```
        .
        .
        .
```